

FIAP - ACLIMAÇÃO
Ciências da Computação

ARTHUR MAZIVIERO FARIA. RM: 573928
JUN UEHARA. RM: 570537
FELIPE DE SOUZA GALLO. RM: 569680
ROBERSON REGUERO LUIZ JUNIOR. RM: 573031
TOMMASO CONCEIÇÃO NAGLIATTI. RM: 572147
MATHEUS MARTINS LACERDA. RM: 570843

SPRINT 3
MODELAGEM LINEAR PARA APRENDIZADO
DE MÁQUINA

Estatística com Python

SÃO PAULO
2026

ARTHUR MAZIVIERO FARIA. RM: 573928
JUN UEHARA. RM: 570537
FELIPE DE SOUZA GALLO. RM: 569680
ROBERSON REGUERO LUIZ JUNIOR. RM: 573031
TOMMASO CONCEIÇÃO NAGLIATTI. RM: 572147
MATHEUS MARTINS LACERDA. RM: 570843

SPRINT 3
MODELAGEM LINEAR PARA APRENDIZADO
DE MÁQUINA

Estatística com Python

Relatório da Sprint 3 apresentado ao curso de Ciências da Computação da Instituição de Ensino Superior FIAP - ACLIMAÇÃO, na disciplina Estatística com Python.

1. Objetivo e visão geral

Este trabalho analisa a variável aleatória gasto familiar sob a hipótese de que ela segue uma Distribuição Normal. Em seguida, ajusta uma Regressão Linear Simples para explicar o gasto familiar a partir da renda familiar. Todo o procedimento foi desenvolvido em Python e estruturado para execução no Google Colab.

Resultados principais

Indicador	Resultado
Probabilidade acima da mediana	44.36% - provável
Probabilidade em média $\pm 2s$	95.45% - quase certo
Equação estimada	$\text{gasto} = 207.9033 + 0.2973 \times \text{renda}$
Coefficiente de determinação	$R^2 = 0.9699$

Base de dados

A base contém 50 famílias e duas variáveis monetárias: renda familiar e gasto familiar. Um identificador diferencia os registros, enquanto a coluna autores_trabalho documenta os nomes completos e RMs do grupo. Essas duas colunas não participam dos cálculos. A análise considera a hipótese de normalidade para a variável gasto familiar.

Variável	Descrição
renda_familiar	Renda total observada para cada família, em reais.
gasto_familiar	Gasto total observado para cada família, em reais.
id_familia	Identificador sequencial sem papel no modelo.
autores_trabalho	Identificação do grupo; excluída dos cálculos.

Critério de classificação dos eventos

Foram adotadas faixas explícitas: raro para probabilidade de até 5%; pouco provável acima de 5% e até 25%; provável acima de 25% e até 75%; e quase certo acima de 75%. Essa convenção foi mantida em todas as classificações para assegurar consistência metodológica.

2. Probabilidade acima da mediana

A mediana divide os dados observados em duas metades. A probabilidade foi estimada por uma Distribuição Normal parametrizada com a média e o desvio padrão amostrais, calculando-se a área à direita da mediana por meio da função de sobrevivência.

Código utilizado

```
gastos = dados['gasto_familiar']
media = gastos.mean()
mediana = gastos.median()
desvio = gastos.std(ddof=1)
prob_acima_mediana = norm.sf(
    mediana, loc=media, scale=desvio
)
```

Cálculo	Resultado
Mediana amostral	R\$ 2.127,00
Média usada na Normal	R\$ 2.011,12
Desvio padrão amostral	R\$ 817,21
$P(X > \text{mediana})$	44.3619%
Classificação	Provável

Interpretação

A probabilidade de um gasto selecionado ao acaso ultrapassar a mediana de R\$ 2.127,00 é de aproximadamente 44,36%. O evento é classificado como provável. O valor não precisa ser exatamente 50%: a mediana usada é a da amostra, enquanto a curva Normal é definida pela média e pelo desvio padrão estimados. Como média e mediana amostrais diferem, a área calculada também se afasta de 50%.

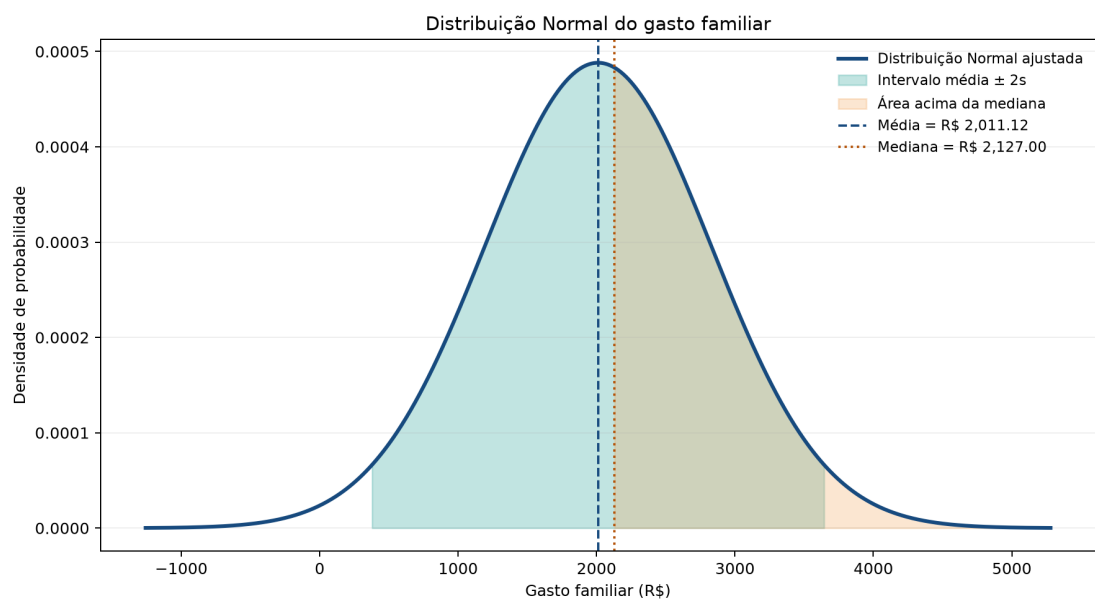


Figura 1 - Distribuição Normal ajustada ao gasto familiar.

3. Probabilidade no intervalo média $\pm 2s$

O segundo evento corresponde à faixa compreendida entre dois desvios padrão abaixo e dois desvios padrão acima da média. Para uma Normal, a probabilidade é obtida subtraindo a função de distribuição acumulada no limite inferior da função acumulada no limite superior.

Código utilizado

```
limite_inferior = media - 2 * desvio
limite_superior = media + 2 * desvio
prob_intervalo = (
    norm.cdf(limite_superior, loc=media, scale=desvio)
    - norm.cdf(limite_inferior, loc=media, scale=desvio)
)
```

Cálculo	Resultado
Média	R\$ 2.011,12
Desvio padrão amostral (s)	R\$ 817,21
Limite inferior (média - 2s)	R\$ 376,69
Limite superior (média + 2s)	R\$ 3.645,55
Probabilidade no intervalo	95.4500%
Classificação	Quase certo

Interpretação

O intervalo vai de R\$ 376,69 a R\$ 3.645,55 e concentra aproximadamente 95,45% da probabilidade do modelo Normal. O evento é classificado como quase certo. Esse resultado coincide com a regra empírica da Normal segundo a qual cerca de 95% dos valores ficam a até dois desvios padrão da média.

A faixa sombreada em azul na Figura 1 representa esse intervalo. A área laranja evidencia, ao mesmo tempo, o evento acima da mediana analisado anteriormente.

4. Modelagem com Regressão Linear

Foi ajustada uma Regressão Linear Simples pelo método dos mínimos quadrados. A variável dependente é o gasto familiar e a variável explicativa é a renda familiar. O modelo assume a forma $\text{gasto} = \text{intercepto} + \text{coeficiente} \times \text{renda} + \text{erro}$.

Código utilizado

```
X = dados[['renda_familiar']]
y = dados['gasto_familiar']
modelo = LinearRegression()
modelo.fit(X, y)
previsoes = modelo.predict(X)
r2 = r2_score(y, previsoes)
rmse = np.sqrt(mean_squared_error(y, previsoes))
```

Parâmetro ou métrica	Estimativa
Intercepto	207.9033
Coefficiente da renda	0.2973
R²	0.9699
RMSE	R\$ 140,36

A reta estimada é: $\text{gasto} = 207.9033 + 0.2973 \times \text{renda}$.

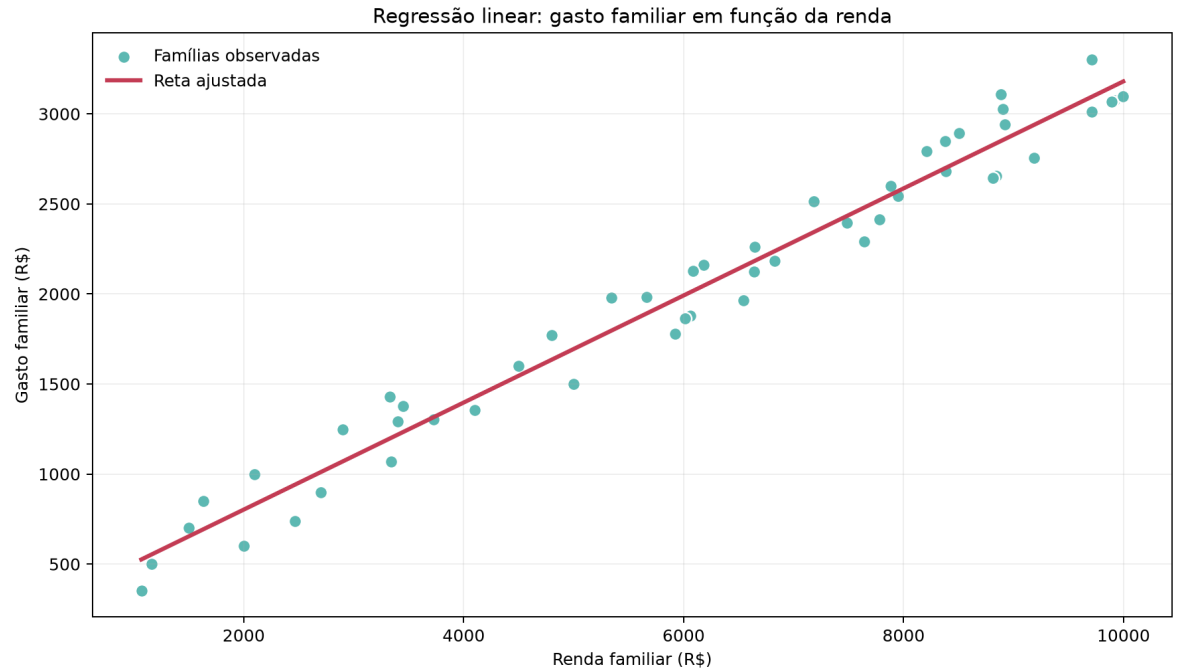


Figura 2 - Observações e reta de Regressão Linear ajustada.

5. Interpretação da regressão

Intercepto

O intercepto estimado é 207.9033. Ele representa um gasto previsto de aproximadamente R\$ 207,90 quando a renda é zero. Como a amostra começa em renda superior a R\$ 1.000,00, essa interpretação é apenas matemática e corresponde a uma extrapolação para fora da faixa observada.

Coeficiente angular

O coeficiente da renda é 0.2973. Portanto, cada aumento de R\$ 1,00 na renda está associado, em média, a um aumento esperado de cerca de R\$ 0.2973 no gasto. Em uma escala mais intuitiva, R\$ 1.000,00 adicionais de renda correspondem a aproximadamente R\$ 297,29 adicionais de gasto previsto.

Qualidade do ajuste

O R^2 de 0.9699 indica que cerca de 96.99% da variação observada no gasto é explicada linearmente pela renda nesta amostra. O RMSE de R\$ 140,36 representa a magnitude típica dos erros de previsão dentro dos dados utilizados no ajuste.

Limites da interpretação

O modelo identifica uma associação linear forte, mas não demonstra que a renda seja a única causa do gasto. Tamanho da família, localização, dívidas e hábitos de consumo podem influenciar o resultado. Além disso, previsões muito além da faixa de renda observada devem ser evitadas.

Relação entre estatística e aprendizado de máquina

A Distribuição Normal fornece uma linguagem probabilística para representar incerteza e classificar eventos. A Regressão Linear usa dados observados para estimar parâmetros e produzir previsões. Assim, a estatística sustenta o aprendizado de máquina ao oferecer medidas de tendência, variabilidade, erro e qualidade do ajuste. O modelo aprende uma regra numérica a partir dos exemplos, enquanto as métricas ajudam a avaliar o quanto essa regra representa os dados.

6. Execução no Google Colab

O notebook challenge_sprint_3.ipynb foi organizado em células sequenciais e reúne a leitura da base, os cálculos, os gráficos e os testes automatizados. A execução no Google Colab utiliza o arquivo dados_familias.csv e não depende de carregamento de código externo. A última célula verifica a consistência dos resultados obtidos.

Bibliotecas

```
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
from scipy.stats import norm
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score
```

Arquivos do projeto

Arquivo	Finalidade
dados_familias.csv	Base de dados utilizada nos cálculos.
analise_estadistica.py	Código Python da análise completa.
Relatório Sprint 3 Modelagem Linear.pdf	Relatório com códigos, gráficos e interpretações.

7. Conclusão

A análise integrou três etapas quantitativas. A chance de o gasto superar a mediana foi classificada como provável; a chance de permanecer a até dois desvios padrão da média foi classificada como quase certa; e a renda explicou aproximadamente 96,99% da variação dos gastos no modelo linear. Os resultados demonstram uma aplicação conjunta de probabilidade, estatística descritiva e aprendizado supervisionado.

8. Validação automatizada

O notebook foi executado integralmente. Sua última célula executou os 10 testes do projeto e 6 testes dos resultados do notebook, totalizando 16 testes aprovados, sem falhas. As verificações abrangem a base de 50 famílias, as estatísticas, as probabilidades, os limites de classificação, a regressão e a consistência com o módulo Python.

Apêndice A - Repositório e código-fonte

O repositório público reúne os arquivos utilizados no desenvolvimento da análise estatística e permite consultar o histórico técnico do projeto.

<https://github.com/felipe-gallo/challenge-sprint-3-estatistica-python>

Código completo de analise_estatistica.py

O código a seguir realiza a leitura e a validação dos dados, os cálculos probabilísticos, a classificação dos eventos, o ajuste da Regressão Linear e a geração dos gráficos.

```
# ARTHUR MAZIVIERO FARIA - RM 573928
# JUN UEHARA - RM 570537
# FELIPE DE SOUZA GALLO - RM 569680
# ROBERSON REGUERO LUIZ JUNIOR - RM 573031
# TOMMASO CONCEIÇÃO NAGLIATTI - RM 572147
# MATHEUS MARTINS LACERDA - RM 570843

"""Análise estatística da Challenge Sprint 3.

Compatível com execução local e com o Google Colab.
A coluna autores_trabalho identifica o grupo e não participa dos cálculos.
Faixas de classificação adotadas: <=5%, <=25%, <=75% e >75%.
"""

from __future__ import annotations

import argparse
from pathlib import Path

import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
from scipy.stats import norm
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score

COLUNA_RENDA = "renda_familiar"
COLUNA_GASTO = "gasto_familiar"

def classificar_evento(probabilidade: float) -> str:
    """Classifica uma probabilidade conforme faixas definidas no projeto."""
    if not 0 <= probabilidade <= 1:
        raise ValueError("A probabilidade deve estar entre 0 e 1.")
    if probabilidade <= 0.05:
        return "raro"
    if probabilidade <= 0.25:
        return "pouco provável"
    if probabilidade <= 0.75:
        return "provável"
    return "quase certo"

def carregar_dados(caminho: str | Path) -> pd.DataFrame:
    """Carrega e valida a base de famílias."""
    caminho = Path(caminho)
    if not caminho.exists():
        alternativas = [Path("dados_familias.csv"), Path("/content/dados_familias.csv"),
                        Path(__file__).resolve().parent / 'dados_familias.csv',
                        Path(__file__).resolve().parents[1] / 'dados/dados_familias.csv']
        caminho = next((item for item in alternativas if item.exists()), caminho)

    if not caminho.exists():
        raise FileNotFoundError(
            "Base não encontrada. No Colab, envie dados_familias.csv antes da execução."
        )

    dados = pd.read_csv(caminho)
    colunas_obrigatorias = {COLUNA_RENDA, COLUNA_GASTO}
    ausentes = colunas_obrigatorias.difference(dados.columns)
    if ausentes:
        raise ValueError(f"Colunas obrigatórias ausentes: {sorted(ausentes)}")
    if dados[list(colunas_obrigatorias)].isna().any().any():
```

```

        raise ValueError("A base contém valores ausentes nas variáveis analisadas.")
    if len(dados) < 2:
        raise ValueError("A análise requer pelo menos duas observações.")
    valores = dados[[COLUNA_RENDA, COLUNA_GASTO]]
    if not all(pd.api.types.is_numeric_dtype(valores[c]) for c in valores):
        raise ValueError("Renda e gasto devem conter apenas números.")
    if not np.isfinite(valores.to_numpy(dtype=float)).all():
        raise ValueError("Renda e gasto devem ser finitos.")
    return dados

def calcular_probabilidades(dados: pd.DataFrame) -> dict[str, float | str]:
    """Calcula as probabilidades solicitadas sob a hipótese Normal."""
    gastos = dados[COLUNA_GASTO]
    media = float(gastos.mean())
    mediana = float(gastos.median())
    desvio_padrao = float(gastos.std(ddof=1))

    if not np.isfinite(desvio_padrao) or desvio_padrao <= 0:
        raise ValueError("A Normal requer desvio padrão positivo e finito.")

    prob_acima_mediana = float(norm.sf(mediana, loc=media, scale=desvio_padrao))
    limite_inferior = media - 2 * desvio_padrao
    limite_superior = media + 2 * desvio_padrao

    prob_intervalo = float(
        norm.cdf(limite_superior, loc=media, scale=desvio_padrao)
        - norm.cdf(limite_inferior, loc=media, scale=desvio_padrao)
    )

    return {
        "media": media,
        "mediana": mediana,
        "desvio_padrao": desvio_padrao,
        "probabilidade_acima_mediana": prob_acima_mediana,
        "classificacao_acima_mediana": classificar_evento(prob_acima_mediana),
        "limite_inferior_2s": limite_inferior,
        "limite_superior_2s": limite_superior,
        "probabilidade_intervalo_2s": prob_intervalo,
        "classificacao_intervalo_2s": classificar_evento(prob_intervalo),
    }

def ajustar_regressao(dados: pd.DataFrame) -> tuple[LinearRegression, dict[str, float]]:
    """Ajusta gasto familiar em função da renda familiar."""
    x = dados[COLUNA_RENDA]
    y = dados[COLUNA_GASTO]
    if dados[COLUNA_RENDA].nunique() < 2:
        raise ValueError("A regressão requer pelo menos duas rendas distintas.")
    modelo = LinearRegression()
    modelo.fit(x, y)
    previsoes = modelo.predict(x)

    metricas = {
        "intercepto": float(modelo.intercept_),
        "coeficiente_angular": float(modelo.coef_[0]),
        "r_quadrado": float(r2_score(y, previsoes)),
        "rmse": float(np.sqrt(mean_squared_error(y, previsoes))),
    }
    return modelo, metricas

def gerar_grafico_distribuicao(
    dados: pd.DataFrame, resultados: dict[str, float | str], destino: Path
) -> None:
    """Gera a curva Normal com mediana e intervalo de dois desvios."""
    media = float(resultados["media"])
    desvio = float(resultados["desvio_padrao"])
    mediana = float(resultados["mediana"])

    inferior = float(resultados["limite_inferior_2s"])
    superior = float(resultados["limite_superior_2s"])

    eixo_x = np.linspace(media - 4 * desvio, media + 4 * desvio, 800)
    densidade = norm.pdf(eixo_x, loc=media, scale=desvio)

    fig, ax = plt.subplots(figsize=(10, 5.5))
    ax.plot(eixo_x, densidade, color="#174A7E", linewidth=2.4, label="Distribuição Normal ajustada")
    ax.fill_between(
        eixo_x,
        densidade,
        where=(eixo_x >= inferior) & (eixo_x <= superior),
        color="#5CB8B2",
        alpha=0.38,
    )

```

```

        label="Intervalo média ± 2s",
    )
    ax.fill_between(
        eixo_x,
        densidade,
        where=eixo_x >= mediana,
        color="#F2A65A",
        alpha=0.28,
        label="Área acima da mediana",
    )
    ax.axvline(media, color="#174A7E", linestyle="--", label=f"Média = R$ {media:,.2f}")
    ax.axvline(mediana, color="#B45309", linestyle=":", label=f"Mediana = R$ {mediana:,.2f}")
    ax.set_title("Distribuição Normal do gasto familiar")
    ax.set_xlabel("Gasto familiar (R$)")
    ax.set_ylabel("Densidade de probabilidade")
    ax.legend(frameon=False, fontsize=9)
    ax.grid(axis="y", alpha=0.2)
    fig.tight_layout()
    fig.savefig(destino, dpi=180, bbox_inches="tight")
    plt.close(fig)

def gerar_grafico_regressao(
    dados: pd.DataFrame, modelo: LinearRegression, destino: Path
) -> None:
    """Gera o gráfico de dispersão e a reta de regressão ajustada."""
    x = dados[COLUNA_RENDA].to_numpy()
    y = dados[COLUNA_GASTO].to_numpy()
    ordem = np.argsort(x)
    x_ordenado = x[ordem]

    x_para_previsao = pd.DataFrame({COLUNA_RENDA: x_ordenado})
    y_previsao = modelo.predict(x_para_previsao)

    fig, ax = plt.subplots(figsize=(10, 5.8))
    ax.scatter(x, y, color="#5CB8B2", edgecolor="white", s=58, label="Famílias observadas")
    ax.plot(x_ordenado, y_previsao, color="#C33C54", linewidth=2.5, label="Reta ajustada")
    ax.set_title("Regressão linear: gasto familiar em função da renda")
    ax.set_xlabel("Renda familiar (R$)")
    ax.set_ylabel("Gasto familiar (R$)")
    ax.legend(frameon=False)
    ax.grid(alpha=0.2)
    fig.tight_layout()
    fig.savefig(destino, dpi=180, bbox_inches="tight")
    plt.close(fig)

def imprimir_resultados(
    probabilidades: dict[str, float | str], regressao: dict[str, float]
) -> None:
    """Exibe resultados de forma organizada no terminal ou no Colab."""
    print("\n1) PROBABILIDADE ACIMA DA MEDIANA")
    print(f"Mediana do gasto: R$ {probabilidades['mediana']:,.2f}")
    print(f"Probabilidade: {probabilidades['probabilidade_acima_mediana']: .2%}")
    print(f"Classificação: {probabilidades['classificacao_acima_mediana']}")

    print("\n2) PROBABILIDADE NO INTERVALO MÉDIA ± 2s")
    print(f"Média: R$ {probabilidades['media']:,.2f}")
    print(f"Desvio padrão amostral: R$ {probabilidades['desvio_padrao']:,.2f}")
    print(
        "Intervalo: "
        f"[R$ {probabilidades['limite_inferior_2s']:,.2f}; "
        f"R$ {probabilidades['limite_superior_2s']:,.2f}]"
    )
    print(f"Probabilidade: {probabilidades['probabilidade_intervalo_2s']: .2%}")
    print(f"Classificação: {probabilidades['classificacao_intervalo_2s']}")

    print("\n3) REGRESSÃO LINEAR")
    print(
        "Equação: gasto_familiar = "
        f"{regressao['intercepto']: .4f} + "
        f"{regressao['coeficiente_angular']: .4f} × renda_familiar"
    )
    print(f"R²: {regressao['r_quadrado']: .4f}")
    print(f"RMSE: R$ {regressao['rmse']:,.2f}")

def executar(caminho_csv: str | Path, pasta_resultados: str | Path) -> dict[str, dict]:
    """Executa a análise completa e salva os gráficos."""
    dados = carregar_dados(caminho_csv)
    probabilidades = calcular_probabilidades(dados)
    modelo, regressao = ajustar_regressao(dados)

```

```

    pasta_resultados = Path(pasta_resultados)
    pasta_resultados.mkdir(parents=True, exist_ok=True)
    gerar_grafico_distribuicao(
        dados, probabilidades, pasta_resultados / "distribuicao_normal.png"
    )
    gerar_grafico_regressao(dados, modelo, pasta_resultados / "regressao_linear.png")
    imprimir_resultados(probabilidades, regressao)
    return {"probabilidades": probabilidades, "regressao": regressao}

def main() -> None:
    parser = argparse.ArgumentParser(description="Challenge Sprint 3 - Estatística")
    parser.add_argument(
        "--arquivo",
        default="dados/dados_familias.csv",
        help="Caminho do arquivo CSV",
    )
    parser.add_argument(
        "--saida",
        default="resultados",
        help="Pasta em que os gráficos serão salvos",
    )
    argumentos = parser.parse_args()
    caminho = Path(argumentos.arquivo)
    if not caminho.exists() and Path('/content').exists():
        from google.colab import files
        print('Selecione dados_familias.csv, entregue junto com este código.')
        enviados = files.upload()
        csvs = [nome for nome in enviados if nome.lower().endswith('.csv')]
        if len(csvs) != 1:
            raise ValueError('Envie exatamente um arquivo CSV.')
        caminho = Path(csvs[0])
    executar(caminho, argumentos.saida)

if __name__ == "__main__":
    main()

```

Apêndice B - Base de dados das famílias

A tabela apresenta os 50 registros utilizados na análise. Cada observação contém o identificador da família, a renda familiar e o gasto familiar. A coluna de autoria presente no arquivo CSV possui finalidade documental e não participa dos cálculos estatísticos.

Família	Renda familiar (R\$)	Gasto familiar (R\$)
1	9.714	3.011
2	3.728	1.305
3	6.062	1.879
4	8.845	2.654
5	8.378	2.849
6	3.338	1.068
7	8.507	2.892
8	7.947	2.543
9	9.915	3.074
10	1.632	849
11	6.825	2.184
12	8.918	2.943
13	4.100	1.357
14	9.184	2.755
15	6.180	2.163
16	9.997	3.099
17	4.500	1.600
18	1.069	353
19	5.925	1.778
20	2.466	740
21	6.083	2.129
22	9.712	3.302
23	7.780	2.412
24	8.383	2.683
25	7.185	2.515
26	7.483	2.395
27	7.640	2.292
28	2.100	1.000
29	2.000	600
30	6.012	1.864

Família	Renda familiar (R\$)	Gasto familiar (R\$)
31	8.902	3.027
32	5.345	1.978
33	8.210	2.791
34	5.662	1.982
35	2.700	900
36	6.546	1.964
37	2.900	1.247
38	9.894	3.067
39	1.500	700
40	5.000	1.500
41	8.885	3.110
42	8.813	2.644
43	3.446	1.378
44	7.881	2.601
45	1.164	501
46	3.401	1.292
47	6.641	2.125
48	3.329	1.431
49	6.648	2.260
50	4.800	1.770